CÓDIGOCLARO · MATERIAL DE ESTUDIO

SQL, MySQL y PL/SQL

Cuaderno DAM/DAW · MySQL 8.4 con InnoDB y Oracle AI Database 26ai · dialectos separados.

Modelo relacional y claves

Objetivo: Traducir requisitos de un módulo DAM a tablas con identificadores y relaciones.

La clave primaria identifica cada fila y la clave foránea mantiene una referencia válida. Una clave compuesta impide duplicar una relación concreta. Modela cardinalidades reales para evitar copiar el nombre del módulo en cada matrícula. Comprueba restricciones con filas válidas e inválidas, no solo con el caso normal.

Código · MySQL 8.4

```
CREATE TABLE alumno (
  id INT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL
);
CREATE TABLE matricula (
  alumno_id INT NOT NULL,
  modulo VARCHAR(60) NOT NULL,
  PRIMARY KEY (alumno_id, modulo),
  FOREIGN KEY (alumno_id) REFERENCES alumno(id)
);
```

La clave primaria hace única e identificable cada fila; la foránea exige una referencia válida a la tabla relacionada. La clave compuesta alumno_id, modulo evita repetir una misma matrícula sin impedir que el alumno curse otros módulos. NOT NULL expresa que la relación debe estar completa. Prueba una matrícula válida y otra con un alumno inexistente: el motor debe rechazar la segunda por la restricción, no por una comprobación manual posterior. La clave primaria compuesta impone la regla en la base incluso si llegan dos peticiones simultáneas. La aplicación puede validar antes para mostrar un mensaje, pero la restricción sigue siendo la garantía final.

Práctica: ¿Qué evita repetir una matrícula del mismo alumno en el mismo módulo?

> Respuesta razonada: La clave primaria compuesta alumno_id y modulo hace única la pareja. La clave foránea exige que el alumno exista; NOT NULL impide una relación incompleta.

Documentación consultada: MySQL 8.4 Reference Manual · Oracle AI Database 26ai · SQL Language Reference

Normalización y dependencias

Objetivo: Reducir duplicidad al guardar datos de alumnos, módulos y profesorado.

Si el nombre del módulo aparece en muchas filas, cambiarlo obliga a modificar varias copias y puede dejar contradicciones. Separa atributos que dependen de claves distintas y enlázalos mediante identificadores. Normalizar no significa dividir por rutina: explica qué dependencia funcional evita cada tabla y verifica que una consulta pueda reconstruir el dato.

Código · MySQL 8.4

```
CREATE TABLE modulo (
  id INT PRIMARY KEY,
  nombre VARCHAR(80) NOT NULL
);
CREATE TABLE matricula (
  alumno_id INT NOT NULL,
  modulo_id INT NOT NULL,
  PRIMARY KEY (alumno_id, modulo_id)
);
```

En este ejemplo, nombre depende del identificador del módulo y no de cada matrícula. Guardarlo repetido puede dejar dos nombres distintos para el mismo módulo después de una actualización parcial. Separar modulo y matricula evita ese cambio múltiple; el JOIN reconstruye la vista cuando se necesita. Antes de normalizar, identifica qué atributo depende de qué clave y conserva las reglas que el sistema debe hacer cumplir. La separación también evita anomalías de inserción y borrado: se puede registrar un módulo antes de matricular a alguien y borrar una matrícula sin perder el nombre del módulo. Las claves mantienen la relación.

Práctica: Si cambia el nombre de un módulo, ¿cuántas filas se actualizan con este diseño?

> Respuesta razonada: Se actualiza una fila de modulo; las matrículas conservan el identificador. Se elimina la anomalía de actualización sin duplicar el nombre en cada inscripción.

Documentación consultada: MySQL 8.4 Reference Manual

SELECT, GROUP BY y HAVING

Objetivo: Calcular medias por módulo separando filtros de fila y filtros de grupo.

WHERE se aplica antes de agrupar; GROUP BY define grupos y HAVING filtra esos grupos ya calculados. Incluye en la agrupación las columnas no agregadas y ordena por el criterio del informe. Escribe primero la población de filas que quieres analizar y luego el agregado para no mezclar convocatorias.

Código · MySQL 8.4

```
SELECT modulo_id, AVG(nota) AS media
FROM resultado
WHERE convocatoria = 'ordinaria'
GROUP BY modulo_id
HAVING COUNT(*) >= 5
ORDER BY media DESC;
```

WHERE reduce filas antes de formar grupos; HAVING filtra el resultado agregado. La consulta conserva una fila por modulo_id y calcula AVG y COUNT dentro de esa misma agrupación. Si se necesita solo una convocatoria, el filtro va en WHERE; si se pide solo módulos con cinco resultados, esa condición depende del agregado y va en HAVING. Ordena por media para que el informe no dependa del orden físico de las filas. COUNT(*) cuenta filas del grupo y AVG ignora valores NULL en la columna agregada. Si el ejercicio incluye notas ausentes, define si se excluyen o se consideran antes de interpretar el promedio.

Práctica: ¿Qué ocurre con un módulo que tiene solo cuatro notas ordinarias?

> Respuesta razonada: No aparece: WHERE conserva las notas ordinarias y HAVING elimina grupos con menos de cinco. AVG se calcula por módulo, no sobre toda la tabla.

Documentación consultada: MySQL 8.4 · SELECT

JOIN y cardinalidad

Objetivo: Conservar alumnos aunque todavía no tengan matrículas.

INNER JOIN conserva pares coincidentes; LEFT JOIN conserva cada fila izquierda y rellena con NULL las columnas derechas sin pareja. Una relación uno a muchos repite la fila de la tabla izquierda. Revisa la cardinalidad y el efecto de los filtros posteriores, porque un WHERE sobre la tabla derecha puede eliminar los nulos.

Código · MySQL 8.4

```
SELECT a.id, a.nombre, m.modulo_id
FROM alumno AS a
LEFT JOIN matricula AS m
  ON m.alumno_id = a.id
ORDER BY a.id, m.modulo_id;
```

Para un alumno sin matrícula, LEFT JOIN conserva una fila del lado alumno y asigna NULL a las columnas de matricula. Un filtro como WHERE m.modulo_id = 3 descarta esas filas y se comporta como coincidencia obligatoria; si hay que mantener alumnos sin matrícula, expresa el criterio en ON o diseña la condición con el resultado deseado. Antes de agregar resultados, cuenta filas por clave para detectar multiplicación uno-a-muchos. Un JOIN puede multiplicar un alumno por cada matrícula encontrada. Si solo interesa saber si existe alguna, una condición EXISTS puede expresar esa intención sin presentar todas las filas relacionadas.

Práctica: ¿Qué ve una consulta para un alumno sin matrículas?

> Respuesta razonada: La fila de alumno permanece y modulo_id vale NULL. Si ese campo se filtra después con WHERE, la fila puede desaparecer; el filtro debe situarse según el resultado que exige el enunciado.

Documentación consultada: MySQL 8.4 · SELECT · MySQL 8.4 Reference Manual

Subconsultas y CTE

Objetivo: Nombrar un conjunto intermedio al construir un informe de módulos con promedio suficiente.

Una subconsulta o un CTE permite separar pasos de una consulta. WITH define un resultado temporal dentro de una sola sentencia, no una tabla persistente. Selecciona solo columnas necesarias y mantén la misma granularidad entre el cálculo y la salida. Un CTE mejora lectura, pero no garantiza por sí solo un plan más rápido.

Código · MySQL 8.4

```
WITH medias AS (
  SELECT modulo_id, AVG(nota) AS media
  FROM resultado GROUP BY modulo_id
)
SELECT modulo_id, media FROM medias
WHERE media >= 7.0;
```

El CTE medias nombra el agregado y deja que la consulta exterior filtre una fila por módulo. No convierte esa relación en tabla permanente ni cambia su granularidad. Una subconsulta puede expresar el mismo cálculo; elige la forma que deja más clara la intención. Para averiguar qué filas alimentan el promedio, ejecuta primero el SELECT interno con un módulo concreto y compara el resultado manualmente. Un CTE puede ayudar a revisar cada etapa por separado. Aun así, el optimizador decide el plan físico; la legibilidad de WITH no debe presentarse como una mejora automática del rendimiento.

Práctica: ¿Por qué alumno_id no forma parte del GROUP BY del informe?

> Respuesta razonada: El informe tiene una fila por módulo. Incluir alumno_id cambiaría la granularidad a una fila por alumno y produciría una media distinta.

Documentación consultada: MySQL 8.4 Reference Manual

DML y parámetros seguros

Objetivo: Cambiar una matrícula por identificador evitando construir SQL con texto de usuario.

INSERT crea filas, UPDATE modifica y DELETE elimina las seleccionadas. Antes de una operación masiva inspecciona el filtro con SELECT; usa parámetros del controlador y una transacción cuando haya varios cambios relacionados. Parámetros protegen la sintaxis frente a inyección, pero no validan la regla de negocio ni corrigen un WHERE demasiado amplio.

Código · MySQL 8.4

```
UPDATE matricula
SET convocatoria = ?
WHERE alumno_id = ? AND modulo_id = ?;
-- El driver enlaza los valores como parámetros.
```

UPDATE y DELETE afectan todas las filas que cumplen su predicado, así que conviene ejecutar un SELECT con el mismo WHERE antes de modificar datos. Los parámetros del controlador separan valores de la sintaxis de SQL; no concatenes texto del usuario. Después revisa filas afectadas: cero puede significar identificador inexistente, mientras que más de una puede indicar un filtro demasiado amplio. Incluye restricciones y transacción cuando varios cambios formen una unidad. El enlace de parámetros no protege contra una regla de negocio incorrecta: hay que comprobar que la persona tiene permiso para modificar esa matrícula. Transacción, autorización y validación resuelven problemas distintos.

Práctica: El identificador del módulo llega vacío. ¿Debe ejecutarse la actualización?

> Respuesta razonada: No. Valida la entrada y comprueba que el filtro apunta a la matrícula concreta. El enlace de parámetros evita inyección, pero la aplicación debe rechazar identificadores incompletos.

Documentación consultada: MySQL 8.4 Reference Manual

Transacciones e InnoDB

Objetivo: Mantener coherente el stock al confirmar una reserva.

Una transacción agrupa operaciones que deben confirmarse juntas o revertirse juntas. En MySQL, el motor de almacenamiento importa: InnoDB ofrece transacciones y aislamiento; no supongas las mismas garantías para cualquier tabla. COMMIT confirma los cambios pendientes y ROLLBACK los descarta. Comprueba las filas afectadas antes de confirmar una reserva y no mezcles esta sintaxis con bloques PL/SQL de Oracle.

Código · MySQL 8.4

```
START TRANSACTION;
UPDATE material SET unidades = unidades - 1
WHERE id = 12 AND unidades > 0;
-- Comprobar filas afectadas antes de confirmar.
COMMIT; -- o ROLLBACK si no se reservó
```

ACID resume propiedades transaccionales: atomicidad, consistencia, aislamiento y durabilidad. En el ejemplo, la condición de stock protege la regla local y la transacción permite revertir si la reserva completa no puede confirmarse. La aplicación debe atender fallos antes del COMMIT y cerrar la operación con ROLLBACK cuando corresponda. InnoDB ofrece niveles de aislamiento que afectan lecturas concurrentes; no asumas que dos sesiones ven el mismo estado sin definir el escenario.

Práctica: La actualización de unidades afecta cero filas. ¿Qué hace el flujo con la operación relacionada y por qué?

> Respuesta razonada: No se confirma la reserva: se revierte la transacción y se informa de que no quedan unidades o de que el material no existe. La condición unidades > 0 evita descontar bajo cero, pero las filas afectadas deciden si la operación tuvo éxito. Si se confirma antes de comprobar, la regla de negocio puede quedar a medias.

Documentación consultada: MySQL 8.4 · transacciones InnoDB

Índices y EXPLAIN

Objetivo: Examinar un índice compuesto según las consultas reales del inventario.

Un índice acelera ciertas búsquedas pero añade coste al insertar y actualizar. En un índice compuesto importa el orden de columnas y los prefijos que puede usar el optimizador. EXPLAIN muestra el plan estimado; mide con datos representativos antes de añadir índices y comprueba si la consulta usa realmente la parte esperada.

Código · MySQL 8.4

```
CREATE INDEX ix_matricula_alumno_modulo
ON matricula (alumno_id, modulo_id);
EXPLAIN
SELECT modulo_id FROM matricula
WHERE alumno_id = 42;
```

Registra el patrón de consulta y el volumen aproximado de filas antes de comparar el plan estimado con una medición controlada.

Un índice compuesto por (alumno_id, modulo_id) está ordenado por su primera columna y puede ser útil para búsquedas que comiencen por alumno_id. No implica que el filtro solo por modulo_id use ese mismo prefijo de forma eficiente. EXPLAIN muestra cómo el optimizador estima ejecutar la consulta; compáralo antes y después con datos representativos. Todo índice ocupa almacenamiento y debe mantenerse al escribir, así que mide también el coste de INSERT y UPDATE. Un índice no corrige una consulta mal construida ni garantiza una lectura rápida para cualquier volumen. EXPLAIN informa del plan estimado; contrástalo con mediciones controladas y la consulta exacta del informe.

Práctica: ¿El mismo índice sirve necesariamente para filtrar solo por modulo_id?

> Respuesta razonada: No necesariamente: el prefijo empieza por alumno_id. Si ese patrón es frecuente, evalúa un índice con modulo_id primero y compara costes de consulta y escritura.

Documentación consultada: MySQL 8.4 · EXPLAIN

PL/SQL y SELECT INTO

Objetivo: Distinguir un bloque de Oracle de una consulta SQL MySQL.

PL/SQL añade variables, flujo y excepciones sobre SQL de Oracle. SELECT INTO asigna exactamente una fila: cero produce NO_DATA_FOUND y varias TOO_MANY_ROWS. Si se esperan varias filas, usa cursor o una operación agregada. La etiqueta del dialecto importa porque MySQL no ejecuta esta sintaxis de bloque.

Código · Oracle PL/SQL 26ai

```
DECLARE
  v_nombre employees.last_name%TYPE;
BEGIN
  SELECT last_name INTO v_nombre
  FROM employees WHERE employee_id = 100;
  DBMS_OUTPUT.PUT_LINE(v_nombre);
EXCEPTION
  WHEN NO_DATA_FOUND THEN DBMS_OUTPUT.PUT_LINE('No existe');
END;
/
```

PL/SQL usa DECLARE, BEGIN y EXCEPTION dentro de Oracle; el slash final es una orden de ciertos clientes como SQL*Plus, no una sentencia del bloque. SELECT INTO asigna una sola fila: ninguna produce NO_DATA_FOUND y más de una, TOO_MANY_ROWS. %TYPE ancla la variable al tipo de columna. Si el filtro puede devolver varios empleados, define una regla única o procesa el conjunto con un cursor; no suprimas la excepción. El ejemplo utiliza sintaxis de Oracle. En MySQL, las variables, el bloque y el tratamiento de errores se escriben de otra forma; mantener el dialecto visible evita mezclar comandos no compatibles.

Práctica: ¿Qué ocurre si el filtro no es único y encuentra varias filas?

> Respuesta razonada: Se produce TOO_MANY_ROWS. SELECT INTO no convierte un conjunto plural en un escalar; cambia la consulta o procesa las filas mediante un cursor.

Documentación consultada: Oracle AI Database 26ai · PL/SQL Language Reference · Oracle AI Database 26ai · SQL Language Reference

Rutinas, cursores y triggers

Objetivo: Elegir una herramienta PL/SQL según se necesite resultado, recorrido o reacción a DML.

Una función devuelve un valor; un procedimiento ejecuta una operación; un cursor permite procesar varias filas y un trigger reacciona a cambios DML. Prefiere SQL set-based cuando expresa la operación completa. Los triggers por fila disponen de :OLD y :NEW; evita lógica oculta que complique comprender los cambios y no confirmes la transacción desde el trigger. MySQL y Oracle tienen rutinas y lenguajes procedurales distintos: etiqueta siempre el dialecto.

Código · Oracle PL/SQL 26ai

```
CREATE OR REPLACE FUNCTION duplicar(p_num NUMBER)
RETURN NUMBER IS
BEGIN
  RETURN p_num * 2;
END;
/
```

La función del ejemplo recibe un número y devuelve su doble con RETURN; un procedimiento se invoca para una acción y puede exponer datos por parámetros OUT. Un cursor se usa cuando hay varias filas, pero una consulta por conjunto suele ser más clara. Un trigger se ejecuta por un evento declarado y puede comparar los valores anterior y nuevo. Mantén estos ejemplos identificados como Oracle PL/SQL; no son sintaxis de MySQL.

Práctica: ¿Qué devuelve duplicar(4), qué haría una función con NULL según la regla definida y cuándo usarías un trigger con :OLD/:NEW?

> Respuesta razonada: La función devuelve 8 y RETURN entrega el resultado al llamador. Para NULL define si se propaga o se rechaza; no inventes una regla. Un trigger se ejecuta al producirse el evento declarado y puede comparar :OLD con :NEW en UPDATE. No lo confundas con un procedimiento invocado de forma explícita.

Documentación consultada: Oracle AI Database 26ai · PL/SQL Language Reference

Fuentes oficiales

MySQL 8.4 Reference Manual · Oracle AI Database 26ai · SQL Language Reference · Oracle AI Database 26ai · PL/SQL Language Reference · MySQL 8.4 · SELECT · MySQL 8.4 · transacciones InnoDB · MySQL 8.4 · EXPLAIN